

Individual Development Journal

COMP710 Team Game Project

Martin Yan 21144088

Week 9 – Project Kickoff

Friday 8 May 2026 – Studio Session

First studio session for the team project. The team met and discussed the game concept. We decided to make a tower defence game where the player draws the path themselves before the enemy wave starts. The idea is that the path drawing phase is the main strategy element – if you draw a longer and more winding path, the enemies take longer to reach the end.

After the meeting the team shared the existing codebase on GitHub. I cloned the repository and spent some time just looking at the folder structure to understand what was already there. There are a lot of files and I was not sure where to start reading, so I just opened each .h file one by one to get a general idea of what classes existed.

Action points from meeting:

- Everyone to read through the existing code before next session
- Assign feature responsibilities once we understand the codebase better
- Start thinking about what each person will implement individually

Week 10–11 – Code Analysis and Bug Discovery

This week was reading and analysis only. I did not change any code yet – the goal was just to understand the existing project well enough to know what to build, and to confirm the cause of anything that looked broken before touching it.

Monday 18 May 2026 – Self-directed study

Spent about two hours reading through all the source files more carefully this time. I made notes about what each class is responsible for:

- Game: singleton, manages scenes, renderer, input, FMOD audio
- SceneGame: the main gameplay scene, handles WASD input for path drawing
- Tilelist: stores all the tiles, manages the path vector and start/end positions
- Tile: individual grid cell, has NextPosition and PrevPosition pointers for linked list
- Pathmaker: just holds the player's current grid position
- DynamicText, AnimatedSprite, ParticleEmitter: all fully implemented but not used anywhere in the scene yet

The most interesting thing I noticed was that Tile has NextPosition and PrevPosition pointers. This forms a linked list along the drawn path, which means once the path is complete you can follow the chain from start to end. I thought this would be useful for an enemy movement system.

Wednesday 20 May 2026

Started testing the existing build to see how it runs. Found some issues immediately:

- The player can walk off the right side of the screen – the boundary check seemed wrong
- When pressing B to undo, the player jumps to a completely wrong position
- Tiles seem to sometimes start with wrong colours before any path is drawn

Started investigating. Found that the Tile constructor is completely empty – none of the member variables are initialised. This explains the random tile colours on startup.

Also traced the undo bug – in SceneGame the code was doing pathmaker->pos.x = pos.y and pathmaker->pos.y = pos.x. The x and y values were swapped. Very simple fix but it took a while to find because I was looking at the Tilelist Undo function first, not the SceneGame code that uses the return value.

Friday 22 May 2026 – Studio Session

Showed the team the bugs I found. Also found the boundary check bug – the MovePosition function was comparing position.x >= rows but x is the column direction so it should be comparing against columns. Since columns = 32 and rows = 18 for a 1280x720 screen, this was limiting horizontal movement to 18 tiles instead of 32.

Also found a bug in Tile::setPrevious where the function was assigning this->PrevPosition = NextPosition instead of the parameter. So the previous pointer in the path linked list was always null. This would cause problems for anything that needed to traverse the path backwards.

Action points:

- Fix all four bugs next week – only analysed and confirmed the causes this week, did not change any code yet
- Start planning the enemy movement feature
- Research how to use the existing DynamicText class

Week 12 – Implementation (Bug Fixes, Enemy System, HUD, Wave System and Game Over)

This was the main implementation week and where the bulk of my work happened. I started by fixing the bugs I had found, then built out the enemy, HUD, wave and game over systems, and finished by adding the missing specification features.

Monday 25 May 2026

Fixed all four bugs:

- Tile constructor now initialises all members (isPath, isStart, isEnd, Connections, PrevPosition, NextPosition)
- Fixed setPrevious to store the correct pointer parameter
- Fixed MovePosition boundary check (swapped rows/columns)
- Fixed Undo position restoration (swapped x/y)

Committed the bug fixes to GitHub separately from any new features so the history is clear.

Research: Enemy movement

Started thinking about how the enemy should move. The tile linked list already exists once the path is drawn, so the enemy just needs to follow NextPosition pointers from the start tile to the end tile.

The main thing I needed to figure out was how to convert tile grid coordinates to screen space. Looking at Tile::Initialise, the formula is:

- screen X = Position.x * tileWidth + tileWidth / 2
- screen Y = Position.y * tileWidth + tileWidth / 2

Where tileWidth = renderer.GetWidth() / columns = 1280 / 32 = 40 pixels. So I need to pass the tile size to the Enemy class when initialising it.

Tuesday 26 May 2026

Started writing the Enemy class. Decided to keep it simple:

- Stores a pointer to current tile
- Each frame calculates target position (centre of next tile in screen space)
- Moves toward target at constant speed
- When close enough, snaps to tile centre and advances pointer
- Sets HasReachedEnd flag when it reaches the end tile

Used ball.png from the assets folder as a placeholder sprite since there is no enemy sprite yet.

Wednesday 27 May 2026

Finished the Enemy class and integrated it into SceneGame. When moving becomes false (player reached end tile), SceneGame starts a spawn timer and creates a new Enemy every 1.5 seconds. Each enemy is initialised with the start tile and the tile pixel size.

Tested it and enemies successfully walk along the drawn path. The snapping logic works well – when the enemy is within 1.5 pixels of the target tile centre it snaps and advances. This prevents the enemy from overshooting and looking jittery.

Research: DynamicText

Read through DynamicText.h and DynamicText.cpp to understand how to use it. The interface is:

- Initialise(renderer, fontPath, fontSize, centred)
- SetPosition(x, y)
- SetText(renderer, string)
- Draw(renderer)

One issue: there is no font file in the assets folder. Did some research and found that on Windows I can point to C:\Windows\Fonts\arial.ttf which should be available on any Windows machine. Added a comment in the code to explain this in case it causes issues on other machines.

Added DynamicText HUD to SceneGame:

- Lives counter at top left (16, 16)
- Wave number below it (16, 48)
- Both update in real time when values change

Thursday 28 May 2026

Implemented the proper wave system. Changed the spawning logic from continuous to wave-based:

- Wave 1 spawns 3 enemies
- Each subsequent wave spawns (wave number × 3) enemies
- When all enemies for a wave are spawned and all cleared, next wave begins automatically
- Wave counter on HUD updates each time

Also added console output for debugging (enemy spawned, enemy reached end, new wave starting).

Implemented the game over state:

- When lives reach zero, set `m_bGameOver` flag to true
- Process function returns early if game over (stops all spawning and movement)
- Display a game over image centred on screen (`defeated_gameover.png` in assets)
- Used Sprite class to load and display the image

Originally I was using `DynamicText` to show “GAME OVER” text, but the team decided an image would look better. Changed the implementation to load a sprite instead. I also added a restart system so pressing R resets the lives, wave, enemies and game state and shows the instructions again.

Additional features from specification review

After reviewing the assignment specification more carefully, identified several required features that were missing from the build: fullscreen mode, a splash screen, and a controls instructions screen. Also identified two further features that would improve the project: FMOD audio integration and a particle effects system using the existing Particle framework that had never been connected. I worked through these across Thursday and Friday.

Fullscreen: one-line fix in `game.cpp` – changed `Renderer::Initialise` from windowed (true) to fullscreen (false). The assignment specification explicitly states the game must run in fullscreen by default.

Splash screen: created `SceneSplash` class (`SceneSplash.h`, `SceneSplash.cpp`). Loads the existing `splash.png` asset, displays it fullscreen, and auto-advances to `SceneGame` after 3 seconds. Can also be skipped by pressing SPACE or ENTER. To support scene transitions, added a public `NextScene()` method to `Game` that increments `m_iCurrentScene`, and a `GetSoundSystem()` getter so scenes can access FMOD without the system pointer being protected. Modified `game.cpp` to register `SceneSplash` as scene 0 and `SceneGame` as scene 1.

Instructions overlay: added `m_bShowInstructions` state to `SceneGame`. On startup the overlay displays a centred list of controls (WASD to draw path, B to undo, ESC to quit) using an array of `DynamicText` objects. The game loop is held until the player presses SPACE to dismiss the overlay. The overlay also reappears on restart.

FMOD audio: used the new `GetSoundSystem()` getter to access the shared FMOD system from `SceneGame`. Loaded three sound files: `music_bg.mp3` (looping background music started at initialise), `sound_wavestart.wav` (plays at the start of each wave), and `sound_gameover.wav` (plays when lives reach zero). Kept a `FMOD::Channel` pointer to the background music channel so it can be stopped on game over and restarted on restart. All sounds are released in the `SceneGame` destructor.

Particle effects: pre-allocated a pool of 40 Particle objects in `SceneGame`, sharing a single `explosion.png` sprite. Added a `SpawnBurst(float x, float y)` method that activates 8 dormant particles from the pool at the given screen position, each with a randomised outward velocity and a lifespan between 0.5 and 0.8 seconds. The burst is triggered when an enemy reaches the end tile or is killed by a tower. Processing and drawing all alive particles is handled in the main Process and Draw loops. Restarting the game resets all particles to inactive.

Enemy HP system: added `m_iHP` and `m_iMaxHP` to `Enemy`, with HP scaling per wave (wave N gives $1 + N \times 2$ HP). Added `TakeDamage(int)` and `IsDead()` methods. Modified `Initialise` to accept an optional `waveNumber` parameter (default 1 for backward compatibility). `SceneGame` now passes `m_iWave` when spawning enemies, and checks `IsDead()` each frame in addition to `HasReachedEnd()`. This prepares `Enemy` for damage from towers without requiring any further changes to `Enemy` itself.

Friday 29 May 2026 – Studio Session

Showed the team the full game loop: draw path → enemies walk along it → wave counter increases → lives decrease when enemy reaches end → game over screen when lives hit zero. The lecturer gave positive feedback on the enemy movement system and suggested making the wave system more visible to the player. The game is now playable end-to-end, which was the goal for this week.

Week 13 – Gold Economy, Testing and Documentation

Monday 1 and Tuesday 2 June 2026 – Gold economy system

With the enemy, wave and HP systems already in place, the next feature I picked up was the gold economy, since the tower system being built by a teammate needs a currency to spend. I implemented it as a self-contained system so that it would not interfere with the tower work happening in parallel.

I started by putting every economy number into a single new header file, `EconomyConfig.h`, so that balancing later only requires editing one file rather than hunting for constants across the code. The values I chose are a starting gold of 300, a base kill reward of 6 gold on wave 1, a per-wave decay of 0.9 with a floor of 2 gold, a 40 second pay window per wave, and a short-path bonus. These are tentative defaults to be tuned during playtesting.

In `SceneGame` I added an `m_iGold` counter and a gold label on the HUD next to the existing lives and wave text. Gold is awarded in the same enemy-death branch that already triggers the particle burst, so killing a normal enemy adds the current kill reward. Two rules shape the income:

- The kill reward decays each wave. Because the enemy count rises every wave (wave number $\times 3$), a flat reward would make the late game produce far too much gold, so the reward is multiplied by 0.9 each wave down to a floor of 2.
- Each wave only pays out for the first 40 seconds. A timer starts when the first enemy of a wave spawns, and kills after the window closes give nothing. This addresses the “tunnel” problem from the design document: a very long path takes longer to clear, so the player runs out of paying time and effectively punishes themselves for an over-long path, without needing any explicit path-length check.

I also added a one-off short-path bonus, paid the moment the player finishes drawing their path, calculated from how far under the par length the path is and capped at 150 gold. This rewards efficient paths and balances out the pay-window penalty for long ones.

For the tower system to interact with the economy, I exposed a small public interface on `SceneGame` – `GetGold()`, `TrySpend(int cost)` which only deducts gold if the player can afford it, `AddGold(int amount)` for relic or event rewards, and `AddGoldStrikeBonus()` for the Pickaxe “Gold Striker” upgrade. Keeping the interface this small means the tower code only depends on a few methods, and I did not have to modify any tower files myself.

While working on the restart logic I noticed that `RestartGame()` reset lives and wave but would have left the player’s gold from the previous run, so I added the gold reset there as well.

One item I left for the teammate handling the boss and relic system: at the moment a boss kill still pays the normal bounty, because `SceneGame` cannot yet tell a boss apart from a normal enemy. The design is for the boss to drop a relic instead of gold, so once an `IsBoss()` check exists on `Enemy` it can be added to the bounty line. I noted this in the handover.

The economy work was put on a separate branch rather than committed straight to main, because it shares `SceneGame.cpp` and `SceneGame.h` with the tower and enemy work, and I wanted my teammate to review the merge rather than risk overwriting their changes.

Wednesday 3 June 2026 – Testing, review and documentation

Did testing to check for any obvious bugs in the features I implemented. Checked:

- Enemy movement on different path lengths and shapes – works correctly
- Wave escalation – correctly increases enemy count each wave
- Lives counter – decrements correctly and triggers game over at zero
- HUD display – updates at the right times

Also reviewed all my code for any memory leaks. Checked that all new/delete calls are paired, and that the enemy vector is fully cleaned up in the SceneGame destructor. Wrote commit messages for all final changes and pushed to GitHub.

Wrote the individual post-mortem document reflecting on the development process, and prepared the developer journal (this document).

Thursday 4 June 2026 – TDD and UML class diagram

The main task this day was writing the Technical Design Document (TDD) and drawing the UML class diagram for the project. Producing the TDD and UML class diagram not only accurately reflected the final architecture of the current system, but also served as an important review and validation of the design iterations the team had worked through earlier in the project.

The TDD covered the target platform, development environment and tools; the required technologies and libraries (SDL2, SDL2_image, SDL2_ttf, OpenGL + GLEW, Box2D, FMOD, ImGui, and the COMP710 framework); key technical challenges and solutions; an architecture overview including the UML class diagram, a state machine diagram for the game flow, and a sequence diagram for a tower attacking an enemy; technical risks and mitigation; and the feature scope written as a MoSCoW list.

For the technical challenges section I described the real systems rather than generic ones – for example the Box2D circle sensor each tower uses to find enemies in range, the tile linked list the enemies follow, and the way the gold economy (pay window plus bounty decay) solves the “tunnel” problem from the design document without an explicit path-length check.

I drew the UML class diagram with SceneGame at the centre, composing the tile grid, path maker, towers, enemies, projectiles, shop and particle system, with GameData as a shared singleton.

Notes for Gold Milestone:

- Need to make sure defeated_gameover.png is included in the assets folder before final build
- Audio files (music_bg.mp3, sound_wavestart.wav, sound_gameover.wav) must be placed in the assets folder – FMOD will fail silently if they are missing, so this must be checked before the Gold build
- Known issue: if arial.ttf is not present at the hardcoded path, HUD text will not display – this should be noted in Known Issues document
- All features I implemented are functional and have been tested

Summary of Individual Contributions

Feature	Status	Files Modified/Created
Bug fix: Tile constructor uninitialised members	Complete	Tile.cpp

Feature	Status	Files Modified/Created
Bug fix: setPrevious wrong pointer	Complete	Tile.cpp
Bug fix: MovePosition bounds check swapped	Complete	SceneGame.cpp
Bug fix: Undo x/y coordinates swapped	Complete	SceneGame.cpp
Enemy movement system	Complete	Enemy.h, Enemy.cpp, SceneGame.h, SceneGame.cpp
HUD display (lives + wave)	Complete	SceneGame.h, SceneGame.cpp
Wave system	Complete	SceneGame.h, SceneGame.cpp
Game over state with image	Complete	SceneGame.h, SceneGame.cpp
Restart system	Complete	SceneGame.h, SceneGame.cpp
Fullscreen mode	Complete	game.cpp
Splash screen with auto-advance and skip	Complete	SceneSplash.h, SceneSplash.cpp, game.h, game.cpp
Instructions overlay (controls display)	Complete	SceneGame.h, SceneGame.cpp
FMOD audio system (background music, wave start, game over)	Complete	game.h, SceneGame.h, SceneGame.cpp
Particle burst effects on enemy events	Complete	SceneGame.h, SceneGame.cpp
Enemy HP system with wave scaling	Complete	Enemy.h, Enemy.cpp
Gold economy: currency, kill bounty with per-wave decay	Complete	EconomyConfig.h, SceneGame.h, SceneGame.cpp
Pay window (anti-tunnel income timing)	Complete	SceneGame.h, SceneGame.cpp
Short-path bonus on path completion	Complete	SceneGame.cpp
Gold spend interface for towers (TrySpend / AddGold / Gold Striker)	Complete	SceneGame.h, SceneGame.cpp
Bug fix: gold not reset on restart	Complete	SceneGame.cpp

Total contribution: 2,161 lines added, 984 lines removed, 22 commits